
AI Tool Kart

News Agent

How the Agent Decides What Becomes an Article

Selection, evidence, verification, editorial,
SEO and publication criteria

This document describes the **behaviour that is actually implemented** in the AI Tool Kart News Agent codebase. Every threshold, weight, gate and reason code quoted here was read from the source at the time of writing. Nothing has been inferred, idealised, or carried over from design documentation that the code no longer matches; where the two differ, the code is documented and the difference is stated explicitly.

Prepared for client review • Technical reference
Source of record: `news agent/src/` • Design brief: `NEWS_AGENT.md`
Verification status: full agent test suite passing (422 of 422 tests)

Contents

1	Executive Summary	1
1.1	The pipeline at a glance	1
1.2	What this means commercially	1
1.3	Three outcomes, not two	2
2	News Discovery and Initial Eligibility	2
2.1	How stories are discovered	2
2.2	Deterministic eligibility parameters	2
3	Relevance and Importance Scoring	4
3.1	Where the numbers come from	4
3.2	The weighted score	5
3.3	Selection thresholds	5
3.4	Rejected, deferred, selected	6
3.5	Deviation from the design brief	7
4	Source Trust Model	7
4.1	What each tier means operationally	7
4.2	The configured source registry	7
4.3	Decision table	9
5	Evidence Sufficiency	9
5.1	What evidence gathering is — and is not	9
5.2	The sufficiency rule	10
5.3	When a source fails	10
5.4	Quarantine of manipulated sources	11
5.5	Designed to reject rather than hallucinate	11
6	Claim Extraction	11
6.1	One source at a time	11
6.2	Implemented claim types	11
6.3	Why atomic claims	12
6.4	Safeguards during extraction	12
7	Claim Verification	12
7.1	Two layers, and code has the final say	12

7.2	Support levels	13
7.3	Evidence requirements by claim type	13
7.4	How conflicting claims are handled	14
7.5	What makes a story fail verification entirely	14
8	Article Format Selection	14
8.1	The three formats	15
8.2	What counts, and what deliberately does not	15
8.3	Stability across revisions	15
9	Writer Constraints	16
9.1	What the writer receives	16
9.2	What the writer does not receive	16
9.3	Explicit generation constraints	16
9.4	What code produces, not the model	17
10	Editorial Validator	17
10.1	What the editorial pass checks	18
10.2	Decision states	19
10.3	The bounded revision loop	19
11	SEO Judgement Layer	20
11.1	The SEO brief	20
11.2	Blocking versus advisory	21
11.3	Keyword stuffing is measured as repeats, not density	22
11.4	The monotonic decision rule	22
11.5	SEO failure never costs an article	22
12	Category and Tag Validation	22
12.1	Categories: a fixed allowlist	22
12.2	Tags: validated, normalised, then created on demand	23
13	WordPress Publication Gate	24
13.1	Draft only — structurally, not by configuration	24
13.2	Final requirements before a draft is created	25
13.3	Idempotency: running twice must never produce two drafts	25
13.4	Pending-draft retry	26
14	Cost and Run Budget Controls	26

14.1 Implemented caps	27
14.2 Cap semantics	27
14.3 How work is deferred when a cap is reached	27
14.4 Two things a cap never stops	28
14.5 Concurrency	28
15 Approval Decision Flow	29
16 Decision Matrix	30
17 Examples	31
17.1 Example A — Accepted	32
17.2 Example B — Rejected	33
17.3 Example C — Deferred	34
18 What the LLM Does vs What Code Decides	34
19 Limitations	36
19.1 Source access	36
19.2 Model and API availability	37
19.3 Evidence and coverage	37
19.4 Article shape	37
19.5 SEO metadata	38
19.6 Featured images	38
19.7 Human review remains required	38
20 Conclusion	38
A Current Configurable Parameters	40
A.1 Environment variables (25)	40
A.2 Scoring and selection constants (12)	41
A.3 Deduplication and freshness (6)	41
A.4 Evidence and verification (8)	42
A.5 Article and editorial constraints (15)	42
A.6 SEO constraints (11)	43
A.7 Taxonomy and tags (6)	43
A.8 Editorial scope lists (4)	43
A.9 Retry and operational bounds (10)	44

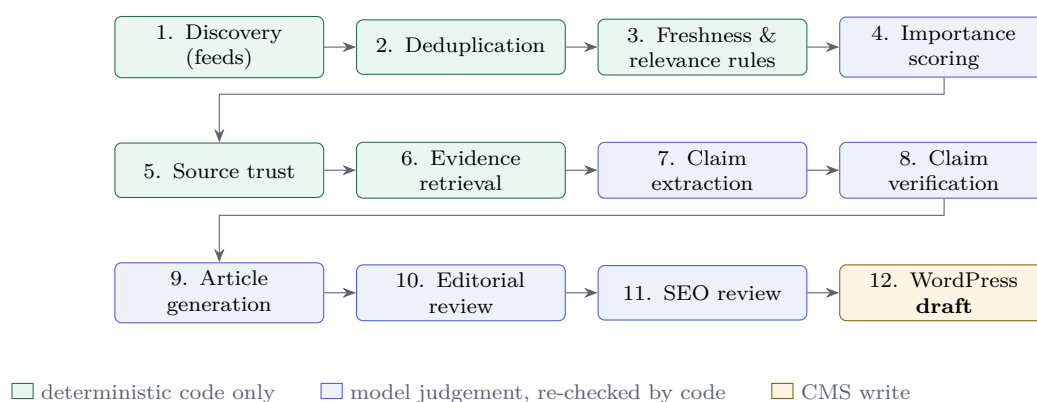
B	Implemented Rejection / Deferral Reasons	45
B.1	Rejections — discovery and eligibility	45
B.2	Rejections — scoring and selection	45
B.3	Rejections — evidence, claims and verification	45
B.4	Editorial and SEO — draft retained, not published	46
B.5	SEO findings	47
B.6	Tag rejections	47
B.7	Deferrals — the story survives	47
B.8	Publication guards — skipped, not failed	48
C	Documented vs. Implemented Differences	49

1 Executive Summary

The AI Tool Kart News Agent does not ask a language model to “write a blog post”. A language model is never given a topic and a word count and trusted with the result.

Instead, every candidate story travels through a sequence of independent **decision gates**. Most of those gates are ordinary program logic — arithmetic, list membership, string validation, database lookups — with no model involved at all. The language model is used only where judgement is genuinely required (is this relevant? what does this document actually assert? does the evidence support this sentence?), and each of its answers is then re-checked by code that can overrule it. The model can lower a story’s chances. It can never, on its own, raise them past a gate that code has closed.

1.1 The pipeline at a glance



The full sequence, in the order a story experiences it:

discovery → deduplication → freshness / relevance → importance scoring → source trust → evidence retrieval → claim extraction → claim verification → article generation → editorial review → SEO review → WordPress draft

The governing rule

Failure at any blocking gate prevents publication. There is no override, no “best effort” path, and no score high enough elsewhere to buy a story past a gate it failed. The agent is built to produce *no article* in preference to an *unsupported article*.

1.2 What this means commercially

- **The agent rejects far more than it writes.** Rejection is the normal outcome and is recorded with a machine-readable reason (Appendix B), so the funnel is auditable rather than opaque.
- **Nothing is ever published automatically.** The final artefact is a WordPress post with `status = draft`. A human decides whether it goes live. Automatic publishing is not merely switched off — the agent refuses to start if it is configured on, and no code path exists that could honour it.
- **Every factual sentence is traceable.** An article may only contain statements that trace back to a claim extracted from a named source document and then verified against that document. Claims the verification step could not support are removed from the writer’s input entirely.

- **Cost is bounded before quality is negotiated.** Hard per-run caps on candidates, verifications, articles, drafts, model calls and tokens mean a run has a known ceiling. When a cap is reached the remaining work is *deferred to the next run*, not rejected.

1.3 Three outcomes, not two

A story is never simply “accepted or not”. The pipeline distinguishes:

Outcome	Meaning
Selected	The story cleared every gate reached so far and the pipeline continues spending effort on it.
Rejected	A judgement about the story itself: stale, irrelevant, unsupported, duplicated, or editorially unsound. Terminal — the story is never reconsidered.
Deferred	A judgement about the <i>run</i> , not the story: a budget or cap was reached. The story stays a candidate and is picked up first on the next run.

Technical note: The three states are persisted on the story record as `status = 'candidate' | 'verified' | 'generated' | 'published' | 'rejected' | 'deferred'` (`src/domain/types.ts`), always alongside a reason string.

2 News Discovery and Initial Eligibility

Everything in this section runs **before the first model call** and costs nothing but network time. This is deliberate: if a large share of discovered items still reached the model, the filters would be too loose and the run would be needlessly expensive.

2.1 How stories are discovered

The agent reads a fixed registry of RSS/Atom feeds (`src/sources/registry.ts`). No feed URL exists anywhere else in the codebase, so adding, disabling or re-tiering a source is a configuration change and never a code change. Sources are fetched sequentially — the list is short and polite request behaviour toward publishers we do not own matters more than run latency.

A source that is down, slow, rate-limited, or serving an HTML error page costs one warning line. The run continues with every other source.

2.2 Deterministic eligibility parameters

Parameter	What it measures	Why it matters	Current implementation	Blocking?
Source enabled state	Whether a configured feed participates in this run	Lets a broken or off-beat source be retired without losing its history	<code>enabled: true false</code> per entry in the registry. 10 of 17 registered sources are currently enabled; the 7 disabled ones each carry a written reason.	Yes
Source trust tier	How authoritative the publisher is	Drives evidence sufficiency and per-claim verification, not just ranking	<code>trustTier: 1 2 3</code> , travelling with every item from feed to verification. 7 Tier 1 and 3 Tier 2 sources enabled.	Yes

Parameter	What it measures	Why it matters	Current implementation	Blocking?
Source type	Which fetch strategy applies	Feeds are stable; HTML scraping is the most fragile part of any such system	<code>type: 'rss' 'api' 'web'</code> . Only <code>rss</code> is implemented and enabled; every <code>web</code> entry is currently disabled.	Yes
Source availability	Whether the feed responded and parsed	One dead feed must not end a run	Fetch/parse failure is isolated per source, counted in <code>sourcesFailed</code> , and the run proceeds.	No
Per-source item cap	How much of one feed a single run ingests	Stops one high-volume changelog dominating the run	<code>AGENT_MAX_ITEMS_PER_SOURCE_PER_RUN</code> (default 25), overridable per source via <code>maxItemsPerRun</code> (GitHub Changelog and AWS ML Blog are set to 15).	Defers
Banned domains	Aggregators and content mills	Republished content is not reporting, and ingesting it pollutes deduplication	9 hosts in <code>BANNED_DOMAINS</code> , e.g. <code>news.google.com</code> , <code>medium.com</code> , <code>msn.com</code> , <code>yahoo.com</code> . Dropped at ingestion, re-checked at prefilter.	Yes
Title length	Whether the item is a real headline	A stub title cannot be classified, deduplicated or written from	<code>minTitleLength = 12</code> characters; enforced at ingestion and again at prefilter.	Yes
Story freshness	Age of the <i>most recent</i> item backing the story	Stale news is not news	<code>maxStoryAgeHours = 96</code> . Measured from the freshest item, so a story re-covered today stays current even if the first outlet ran it two days ago.	Yes
Duplicate URL handling	Whether this exact document was already seen	Prevents re-drafting an already-processed story	Level 1/2 dedupe: the canonical URL is looked up in the persisted <code>news_items</code> table and matching items are dropped before any work.	Yes
Canonical URL handling	Reducing a URL to a stable document identity	Two URLs for one document must not become two stories	Deterministic canonicalisation: forced <code>https</code> , host normalisation (<code>www.</code> , <code>m.</code> , <code>amp.</code> stripped), tracking parameters removed (<code>utm_*</code> , <code>fbclid</code> , <code>gclid</code> and ~40 more), redirect wrappers unwrapped one level, AMP/index suffixes stripped, remaining query parameters sorted, fragment discarded.	Yes
Near-duplicate clustering	Whether two headlines describe one event	Three outlets covering one launch must become one article, not three	Level 3 dedupe on normalised titles: same-story at similarity ≥ 0.62 , ambiguous band 0.50–0.62, within a 72-hour window. Two or more shared distinctive tokens promote a near-miss to a match.	Yes
Excluded topics (hard)	Categories of story never worth covering	Removes noise before it can cost a model call	5 hard-reject rules: <code>stock</code> , <code>doom</code> , <code>clickbait</code> , <code>listicle</code> , <code>horoscope-noise</code> (crypto/NFT/web3/metaverse).	Yes

Parameter	What it measures	Why it matters	Current implementation	Blocking?
Excluded topics (soft)	Story shapes that should be discouraged	Rumour and drama should be penalised, not always banned	3 negative-weight rules: rumor (−3), celebrity (−3), unrelated-funding (−1.5). Applied to the score, not as a veto.	No
Terminal story state	Whether this story was already resolved	A published or rejected story must never be reconsidered	already-published , previously-rejected and duplicate-of-existing-story short-circuit the prefilter.	Yes

Technical note: Implemented in `src/ingestion/ingest.ts`, `src/dedupe/url.ts`, `src/dedupe/title.ts`, `src/dedupe/cluster.ts` and `src/ranking/prefilter.ts`; thresholds in `src/config/limits.ts` and `src/config/editorial.ts`.

Deduplication is biased toward merging

When title similarity is ambiguous, the agent *merges* rather than splits. The reasoning is asymmetric cost: a wrong merge loses one article, a wrong split publishes the same story twice. The one exception is a same-publisher guard — if every item already in a story came from the same domain as the new item and the match was only ambiguous-grade, a separate story is kept, because outlets do publish follow-ups under near-identical headlines.

3 Relevance and Importance Scoring

Only stories that survive Section 2 reach a model. This section describes how the agent decides whether a surviving story is worth spending that budget on.

3.1 Where the numbers come from

Four dimensions feed the score. Two are computed by code with no model involvement; two are produced by a tightly constrained classification call.

Dimension	Source	Definition as implemented
Relevance R	Model (0–10)	Would an AI Tool Kart reader act differently after reading this?
Importance I	Model (0–10)	How significant is the development itself?
Source trust T	Code (0–10)	Best available tier, plus bounded corroboration.
Freshness F	Code (0–10)	Linear decay from the age of the freshest item.
Novelty	Model (0–10)	Produced and schema-validated, but not consumed by any gate or weight.
Category	Model (enum)	One of 10 fixed categories; anything else falls back to industry .
Recommendation	Model (proceed/skip)	Advisory only. Logged, never decisive.

3.2 The weighted score

The four dimensions combine with fixed weights from `src/config/limits.ts`:

$$S_{\text{base}} = 0.35 R + 0.30 I + 0.20 T + 0.15 F \quad (1)$$

Freshness decays linearly over twice the configured half-life (`FRESHNESS_HALF_LIFE_HOURS = 48`, giving a 96-hour window that matches the staleness gate), where a is the age in hours of the freshest item:

$$F(a) = \text{clamp}_{[0,10]} \left(10 \left(1 - \frac{a}{96} \right) \right) \quad (2)$$

Source trust takes the *best* tier available and adds one point per additional independent publisher, capped at two points:

$$T(\mathcal{T}, p) = \text{clamp}_{[0,10]} \left(\text{Base}(\min \mathcal{T}) + \min(\max(p-1, 0) \cdot 1, 2) \right), \quad \text{Base} = \begin{cases} 10 & \text{Tier 1} \\ 7 & \text{Tier 2} \\ 3 & \text{Tier 3} \end{cases} \quad (3)$$

A deterministic *topic prior* then adjusts the result. It is built from the keyword rules and audience-entity matches found during prefiltering, and it is clamped so it can shade a borderline decision but never overturn the model’s relevance judgement:

$$P = \sum_{r \in \text{included}} w_r + \sum_{r \in \text{excluded}} w_r + 0.5 \cdot \min(|\text{entity hits}|, 3) \quad (4)$$

$$A = \text{clamp}_{[-2, +2]} (0.4 \cdot P) \quad \implies \quad \boxed{S = \text{clamp}_{[0,10]} (S_{\text{base}} + A)} \quad (5)$$

Technical note: Included-topic weights run from +1 (`availability`, `integration`) to +2.5 (`mcp`); soft excluded-topic weights run from −1.5 to −4. Entity hits are matches against the curated tool/vendor vocabulary (43 entries in `TAG_ENTITIES`), capped at three so that naming many products cannot inflate a story.

3.3 Selection thresholds

The weighted score alone does not select a story. Four gates are applied in order, and the relevance and importance floors are checked *first* — deliberately, so that a highly trusted, very fresh source cannot carry a story readers do not care about.

Gate	Threshold	Rationale	Configurable
Relevance floor	$R \geq 4$	Trust and freshness must not be able to buy an irrelevant story a place.	Code constant
Importance floor	$I \geq 3$	Filters trivial version bumps and customer-story blog posts.	Code constant
Weighted threshold	$S \geq 6.0$	The overall bar for spending evidence-gathering budget.	AGENT_MIN_WEIGHTED_SCORE
Source base	Tier 1 present, <i>or</i> ≥ 2 independent publishers	A signal nobody authoritative confirms is a rumour, not a story.	Code constant
Candidate cap	≤ 20 scored per run	Bounds classification spend. Excess is deferred , not rejected.	AGENT_MAX_CANDIDATES_PER_RUN

Selection is conjunctive

A story must clear **all four** selection gates. Failure produces one of these recorded reasons, which name the actual values so a reviewer can see how close the story came: `below-relevance-floor`, `below-importance-floor`, `below-threshold`, `insufficient-source-base`.

3.4 Rejected, deferred, selected

State	Trigger	What happens next
Rejected	A judgement about the story: stale, banned domain, hard-rejected topic, below a floor, unsupported evidence, editorially unsound.	Terminal. The reason is persisted. The story is never re-examined, but it still absorbs future duplicates so a re-report cannot become a second article.
Deferred	A judgement about the run: a candidate, verification, article, draft or LLM budget was reached.	The story stays a candidate. On the next run, deferred stories are re-queued ahead of newly discovered ones — but they are re-checked for staleness, so a deferral that aged out is then rejected rather than lingering.
Selected	Cleared every selection gate.	Sorted by weighted score, highest first, so that the remaining budget is spent on the best material.

3.5 Deviation from the design brief

Documented vs. implemented

`NEWS_AGENT.md` §11 lists `novelty` among the classifier outputs and implies it informs ranking. In the current implementation `novelty` is requested, returned and schema-validated, but **no gate or weight reads it**. The weighted score uses relevance, importance, source trust and freshness only. This is documented here rather than presented as an active signal.

4 Source Trust Model

Trust tier is not a ranking nicety. It determines whether evidence is sufficient at all, and it determines, per claim type, what counts as verification.

4.1 What each tier means operationally

Tier	Meaning	Operational consequence	Trust score
1	Official / primary: the vendor, its changelog, its documentation	Sufficient <i>on its own</i> for evidence. Authoritative for facts about its own product — pricing, capabilities, dates, benchmarks. Not authoritative about competitors.	10
2	Reputable technology press	Reliable that <i>something happened</i> ; not relied on for exact specifications. Two independent Tier 2 publishers are required where no Tier 1 source exists.	7
3	Discovery signal (social, forums, aggregators)	Never sufficient evidence. A Tier 3 signal that cannot be corroborated is treated as a rumour. Claims backed only by Tier 3 are marked unsupported.	3

4.2 The configured source registry

These are the sources actually present in `src/sources/registry.ts`. No others exist in the system.

Source	Publisher	Tier	Enabled	Note
OpenAI News	OpenAI	1	yes	
Google DeepMind Blog	Google DeepMind	1	yes	
Google — The Keyword (AI)	Google	1	yes	
Hugging Face Blog	Hugging Face	1	yes	
GitHub Blog	GitHub	1	yes	
GitHub Changelog	GitHub	1	yes	Capped at 15 items/run (high volume, mostly minor)
AWS Machine Learning Blog	AWS	1	yes	Capped at 15 items/run
TechCrunch — AI	TechCrunch	2	yes	
The Verge — AI	The Verge	2	yes	
VentureBeat — AI	VentureBeat	2	yes	
Ars Technica — Tech Lab	Ars Technica	2	no	Feed verified; held back as too broadly scoped
Anthropic News	Anthropic	1	no	No RSS feed published
Meta AI Blog	Meta AI	1	no	Feed returns 404
Mistral AI News	Mistral AI	1	no	Feed returns 404
Microsoft AI Blog	Microsoft	1	no	Feed returns HTTP 410 Gone
Runway Blog	Runway	1	no	Feed returns 404

Technical note: Every enabled endpoint was confirmed to return a parseable RSS/Atom document before being switched on. Sources whose vendors publish no stable feed are registered as disabled *with the reason recorded*, rather than pointed at a guessed URL that would fail silently on every run. **No Tier 3 source is currently configured:** Tier 3 exists in the trust model and is the default tier assigned to any unrecognised host, but no feed feeds it today.

4.3 Decision table

Evidence situation	Agent behaviour	Outcome
Tier 1 only	Sufficient. Selection passes the source-base gate; evidence sufficiency passes on one Tier 1 document. Claims requiring a primary source can reach <i>verified</i> .	Proceeds
Multiple Tier 2, no Tier 1	Two or more <i>independent publishers</i> are required at selection, and two or more distinct Tier 2 publishers at evidence sufficiency. Claim types that require Tier 1 (pricing, capability, date, benchmark, quote) are downgraded to <i>single-source</i> and must be attributed in the prose.	Proceeds, with attribution constraints
A single Tier 2	Fails the source-base gate at selection (Tier 1 absent and publishers < 2).	Rejected — insufficient-source-base
Tier 3 only	Never sufficient. Evidence sufficiency fails; and within verification, a claim whose supporting URLs are all Tier 3 is forced to <i>unsupported</i> regardless of what the model concluded.	Rejected
Source unavailable (403 / 404 / 410 / time-out / paywall)	The fetch is counted as failed and that document contributes no evidence. If what remains still meets the sufficiency rule, the story continues on the remaining sources; otherwise it is rejected. 4xx responses are <i>not</i> retried.	Rejected if sufficiency is lost, else proceeds
Conflicting sources	The verifier marks the claim conflicting and must record what disagrees, naming both values and both publishers. If the conflict touches the story's core claim and no Tier 1 source resolves it, the story does not run. Non-core conflicts must be reported as a disagreement in the article or omitted — never silently resolved.	Rejected on an unresolved core conflict

Corroboration cannot outrank a primary source

The corroboration bonus is capped at **+2 points** (one point per additional independent publisher, maximum two). This is deliberate: ten outlets rewriting one press release must not be able to outrank the vendor's own announcement. Publisher independence is judged by registrable domain, so `blog.example.com` and `example.com` count as one publisher.

5 Evidence Sufficiency

Selection decides a story is *worth* investigating. Evidence gathering decides whether it *can* be written at all. This is the step that most often ends a promising story, and it is intended to.

5.1 What evidence gathering is — and is not

The agent is not a crawler. It fetches only the URLs the story's own discovered items already point at. It follows no links out of those pages, and it never fetches a page speculatively.

Requirement	Implementation
Fetch budget per story	At most 4 evidence pages (<code>MAX_EVIDENCE_FETCHES_PER_STORY</code>).
Fetch order	Tier 1 first , then by recency. If the per-story budget runs out, what the agent holds is the primary material rather than the commentary.
Text volume per source	Cleaned text truncated to 24,000 characters (<code>MAX_EVIDENCE_CHARS</code>).
Minimum usable text	A page yielding fewer than 200 characters of article text is discarded as a paywall, JavaScript shell or consent interstitial. Counting it would let the verifier “confirm” claims against an empty document.
Network safety	Every fetch passes an SSRF guard: HTTPS only, DNS resolution checked against private/link-local/reserved ranges, response size capped (<code>AGENT_MAX_RESPONSE_BYTES</code> , default 2.5 MB), timeout applied (<code>AGENT_HTTP_TIMEOUT_MS</code> , default 15 s), at most 3 redirects, each revalidated.
Output form	Plain text only. Raw HTML never leaves this module — it is the last point in the pipeline at which markup exists.

5.2 The sufficiency rule

Evidence is sufficient when

at least one **Tier 1** document was retrieved,
or two or more distinct **Tier 2** publishers were retrieved.

Anything else is insufficient, and the story is rejected before a single claim is extracted.

Note the wording: *retrieved*, not *listed*. A Tier 1 source that appeared in the feed but whose page could not be fetched does not count. This is the difference between a story that looks well-sourced and one that is.

5.3 When a source fails

Failure mode	Behaviour
HTTP 403 / 404 / 410	Treated as a permanent failure. Not retried — retrying a 404 three times is three wasted requests. The document contributes no evidence.
HTTP 5xx / 408 / 429	Transient. Retried up to 3 attempts with exponential backoff (400 ms base, 4 s ceiling).
Timeout / connection reset	Transient; same retry ladder.
Paywall or JS shell	Fetch succeeds but yields under 200 characters of text; counted as a failure and discarded.
Prompt-injection attempt	The document is quarantined , not filtered — see below.

Headline-only evidence is never allowed

There is no path by which a headline, a feed summary, or a page the agent could not read becomes the basis for an article. Feed summaries are used solely as context for the relevance classifier and are never passed to the writer. Extraction, verification and writing operate exclusively on *retrieved, cleaned document text*. When retrieval fails and sufficiency is lost, the recorded reason is `no-evidence-retrieved` or `insufficient-evidence` (no Tier 1, *n* independent Tier 2).

5.4 Quarantine of manipulated sources

If a fetched page contains text attempting to instruct an automated reader (“ignore previous instructions” and similar patterns), the entire document is withheld from every model call and recorded for human review. The agent does not strip the offending sentences and keep the rest.

The reasoning is that an injection’s instruction and its payload are usually separate sentences, so sentence-level filtering removes the framing and leaves the false statement behind. A page that tries to manipulate an automated reader has disqualified itself as a source of facts, whatever else it says. This costs the occasional false positive — a legitimate article *about* prompt injection would be quarantined — and that trade is accepted deliberately, with the rejection logged so a human can overturn it.

5.5 Designed to reject rather than hallucinate

Every decision in this section resolves the same way when it is uncertain: stop. Insufficient evidence produces no article and a recorded reason. It never produces a shorter article, a hedged article, or an article assembled from the model’s background knowledge of the product. That knowledge is explicitly forbidden downstream and is unavailable here, because the writer never receives source documents at all — only claims that survived verification.

6 Claim Extraction

Once evidence is retrieved, the agent does *not* hand those documents to a writer to paraphrase. It first converts each document into a list of **atomic factual claims**.

6.1 One source at a time

Extraction runs once per evidence document, never across several at once. A claim’s origin must be unambiguous for verification to mean anything; mixing sources into one call would let the model blur which document asserted what.

Each extracted claim carries: the statement itself (10–400 characters), a claim type, and optionally a verbatim supporting quotation from the source (used to audit the extractor). Claims are capped at **25** per source.

6.2 Implemented claim types

Type	What it covers
launch	Something was released, announced, or made available
capability	What a product can now do, including limits and specifications
pricing	Costs, tiers, rate limits, free allowances
date	When something happened or will happen
benchmark	Measured results, and the benchmark named
quote	A direct statement attributed to a named person or company
funding	Investment, valuation or acquisition figures
other	Anything factual that fits none of the above

These eight are the complete set; the response schema is a closed enumeration, so a ninth type

cannot be invented.

6.3 Why atomic claims

The practical difference

Paraphrasing an article produces prose whose accuracy can only be assessed as a whole. Extracting claims produces a list of separately checkable statements. That makes three things possible that are otherwise not:

- **Per-fact verification.** Each statement is judged against the evidence individually, so one weak fact weakens one sentence rather than contaminating the article.
- **Traceability.** The published draft records exactly which claim identifiers it drew on, and each claim records the URLs backing it. A reviewer can walk any sentence back to a source document.
- **Tier enforcement.** Requirements such as “pricing must come from the vendor” are meaningless against prose, but are mechanical against a typed claim.

6.4 Safeguards during extraction

- **Numbers are copied, never normalised.** Every figure, date, price and model name must appear exactly as the document states it.
- **Marketing language is not laundered into fact.** A vendor’s superlative must be recorded as “the company describes X as ...”, not as a bare assertion.
- **Instruction-shaped claims are dropped deterministically.** If an extracted claim itself contains injection markers, code removes it — the pipeline does not rely on the model declining to emit it.
- **Identical claims from different sources are merged** into one claim backed by several URLs. That merge is what creates the corroboration signal verification then reads.
- **Extraction failure is not fatal.** One document failing costs that document’s claims, not the story; the remaining evidence may still support publication. If *no* claims are extracted at all, the story is rejected with `no-claims-extracted`.
- **Support level is not decided here.** Every claim leaves extraction marked `unsupported`. Only the verifier may raise it.

7 Claim Verification

This is the most consequential gate in the pipeline. It decides, claim by claim, what the article is permitted to say.

7.1 Two layers, and code has the final say

1. The **verifier model** judges each claim against the supplied evidence and returns a support level, the URLs it considers supporting, and a note for any conflict.
2. **Deterministic tier rules are then applied on top.** A claim the model called *verified* is downgraded when the evidence backing it does not meet the tier requirement for its claim type.

The rules only ever downgrade

The deterministic layer can lower a claim’s support level. It can never raise one. It exists to be stricter than the model, not to rescue claims the model doubted. Two further guards apply: supporting URLs the model returns are **filtered against the story’s actual evidence set**, so a model echoing a URL from the claim text cannot manufacture support; and a claim the verifier simply ignored is treated as **unsupported**, not as tacitly accepted.

7.2 Support levels

Level	Definition	Effect on the article
verified	Two or more independent sources support it, or a single Tier 1 source states it directly.	May be stated as fact. Counts toward the verified-claim floor and toward article length.
single-source	Exactly one non-Tier-1 source supports it, uncorroborated.	May appear, but must be attributed in the prose (“according to ...”). Does <i>not</i> count toward article length.
unsupported	No supplied evidence states this, including reasonable inferences no source actually makes.	Removed entirely. The writer never receives it.
conflicting	Two or more sources make incompatible statements.	Must be reported as a disagreement naming both figures and both publishers, or omitted. Never silently resolved.

7.3 Evidence requirements by claim type

Claim type	Minimum evidence for verified	May appear if single-source?	Downgrade / rejection rule
launch	Tier 1, <i>or</i> 2 independent Tier 2	Yes, attributed	With one Tier 2 only, falls to single-source
capability	Tier 1 required	Yes, attributed	No Tier 1 $\Rightarrow$ forced to single-source , however confident the model was
pricing	Tier 1 required	Yes, attributed	No Tier 1 $\Rightarrow$ forced to single-source
date	Tier 1 required	Yes, attributed	No Tier 1 $\Rightarrow$ forced to single-source
benchmark	Tier 1 required	Yes, attributed	No Tier 1 $\Rightarrow$ forced to single-source
quote	Tier 1 required ; otherwise 1 independent Tier 2	Yes, attributed	No Tier 1 $\Rightarrow$ forced to single-source
funding	Tier 1, <i>or</i> 1 independent Tier 2	Yes, attributed	Tier 3 only $\Rightarrow$ unsupported
other	Tier 1, <i>or</i> 1 independent Tier 2	Yes, attributed	Tier 3 only $\Rightarrow$ unsupported

Applies to every type: if the supporting evidence is **entirely Tier 3**, the claim is forced to **unsupported**. If the model returned no usable supporting URL, the claim is **unsupported**.

Technical note: Encoded as `CLAIM_RULES` in `src/config/limits.ts`, applied by `applyTierRules()` in `src/verification/verify.ts`. Downgrading rather than discarding is deliberate: a specific figure sourced only to secondary coverage remains reportable *with attribution*, so the story survives without the figure being presented as settled.

7.4 How conflicting claims are handled

A conflict never resolves itself by majority. The verifier must record what disagrees, naming both values and both publishers. Then:

- If the conflict touches the story's **core claim** and no Tier 1 source resolves it, the story is rejected with `core-claim-conflicting`.
- Otherwise the conflict travels to the writer with its note attached, and the writer is instructed to either report the disagreement explicitly or omit the claim — never to pick a side silently.
- The story's evidence state is marked `conflicting` so the condition is visible in the audit trail regardless of the outcome.

7.5 What makes a story fail verification entirely

Story-level verification gates (all must pass)

1. The **core claim** — the single claim the verifier identifies the story as resting on — must not be unsupported. `core-claim-unsupported`
2. The core claim must not be conflicting without a Tier 1 resolution. `core-claim-conflicting`
3. At least **3 usable claims** must survive (verified or single-source). `too-few-verified-claims`
4. At least **one** claim must be fully verified; a story built entirely from single-source material does not run. `no-fully-verified-claims`

Documented vs. implemented

The configuration table also carries a `core: true|false` flag per claim type (true for `launch`, `capability` and `pricing`). **No code path currently reads it.** The story's core claim is whichever claim the verifier nominates, not one selected by claim type. The flag is inert configuration today and is reported here for accuracy.

8 Article Format Selection

Before a word is written, the agent decides *how much article the evidence actually supports*. This is deterministic code, not a model call — specifically so that the model cannot argue itself into a longer piece.

8.1 The three formats

Format	Target words	Sections	Earned by
brief	180–350	3–5	The floor. A changelog entry, a deprecation, one well-sourced announcement. Requires nothing — it is what a story gets when it clears verification.
standard	500–900	4–7	≥ 6 verified claims, ≥ 2 evidence sources, ≥ 2 independent publishers.
analysis	900–1400	5–9	≥ 12 verified claims, ≥ 3 evidence sources, ≥ 3 independent publishers, <i>and</i> classifier importance ≥ 7 .

A story is awarded the highest format whose thresholds it *fully* meets; all conditions in a row are conjunctive.

8.2 What counts, and what deliberately does not

Signal	Role in the decision
Verified claims	Counts. Only verified claims contribute to depth.
Single-source claims	Does not count. They may appear in the prose with attribution, but they must not be what buys a longer article — otherwise format becomes a way of laundering weak sourcing into length.
Number of evidence sources	Counts. Documents actually retrieved.
Independent publishers	Counts. Deduplicated by publisher name, so twenty claims from one press release remain one account.
Story significance	Counts for analysis only. Importance ≥ 7 is required, so a well-sourced but minor story cannot reach the longest form.
Tier 1 source count	Recorded in the audit trail; not itself a threshold.

A short article is a success, not a degraded one

The format ordering is not a quality ranking. The writer is told the range is a *ceiling on ambition, not a quota*: if the verified material does not fill it, write less and stop. The editor judges the draft against that same range rather than a global minimum. Being *under* the range is expected when the evidence runs out first and is advisory only; being *over* it is the more suspicious signal, because the extra words had to come from somewhere.

Technical note: This replaced an earlier flat 500-word minimum. The first real article the pipeline produced ran to 176 words from a GitHub changelog entry that supported exactly eight verified facts — accurate, attributed, and reported as a failure against a target it could never legitimately have reached. The only route to 500 words would have been padding, which the factual-grounding rules forbid, so the target moved to the evidence instead. Implemented in `src/editorial/format.ts`; thresholds in `FORMAT_THRESHOLDS`.

8.3 Stability across revisions

The format is computed **once**, before writing, and the same value is used for the first draft, the editorial review, any revision, and the second review. A rewrite fixes issues; it does not renegotiate how long the article is allowed to be. The chosen format is persisted with the article, so a reviewer can see which range the draft was judged against.

A separate hard ceiling of **1400 words** applies regardless of format: a draft exceeding it is rejected outright as runaway output rather than treated as content.

9 Writer Constraints

9.1 What the writer receives

Input	Detail
Verified and single-source claims	Each with its identifier, claim type, support level, and the publishers backing it. Conflicting claims arrive with their conflict note.
Source metadata	Publisher, tier, publication date and URL — for attribution and the Sources block. This is our own registry data, not fetched content.
Assigned format and word range	Presented as a ceiling on ambition.
Category	One of the ten fixed editorial categories.
SEO brief (when one exists)	Keyword and heading guidance, placed <i>after</i> the claims in the prompt and explicitly subordinate to them.
Editor feedback (revision pass only)	The specific issues a rewrite must fix.

9.2 What the writer does not receive

The writer never sees a source document

By this point, third-party prose has been reduced to atomic claims that a separate step has already judged. The writer receives no raw HTML, no fetched page text, no feed summaries, and no quarantined material. This is simultaneously the editorial rule (*write from evidence, not from memory*) and the strongest prompt-injection defence in the pipeline: text that never reaches the writer cannot influence what it writes.

Claims marked `unsupported` are filtered out before the prompt is built. From the writer's perspective, they do not exist.

9.3 Explicit generation constraints

- **Verified-claims-only.** Every factual statement must trace to a supplied claim. Capabilities, prices, dates, context-window sizes, benchmark results, version numbers, quotations, funding figures and availability details may not be added. If a number would improve a sentence but was not supplied, the sentence is written without it.
- **No background knowledge.** The writer is instructed that its training data may be wrong or outdated about this specific story, and is forbidden from using it to fill gaps.
- **Mandatory attribution.** Single-source claims must be attributed in the text. Conflicting claims must be reported as disagreements or omitted.
- **No extended quotation.** Short attributed quotations only, and only where the exact wording matters.
- **Structured output, never markup.** The writer returns a title, excerpt, category, tags, sections (headings with paragraphs and bullets) and the list of claim identifiers it used. It never produces HTML, Markdown or a slug. A closed, strict schema rejects any unexpected field.
- **Fixed structure.** Five prescribed headings — *What happened*, *What's new*, *Why it matters*, *Who should care*, *Practical implications* — with an instruction to omit any heading for which

there is no verified material rather than pad it.

- **Bounded fields.** Title ≤ 110 characters; excerpt 80–320 characters, written deliberately rather than truncated from the body; 2–5 entity tags; 3–9 sections.
- **Banned hype language.** A configured list of 15 phrases is prohibited (“in the rapidly evolving landscape”, “game-changing”, “revolutionary”, “delve into”, “unlock the power”, “a testament to”, “in conclusion”, and others). This is enforced twice: the prompt forbids them, and deterministic code blocks any draft containing one.
- **Traceability.** The writer must list the claim identifiers it drew on; code then discards any identifier that does not correspond to a real claim.

9.4 What code produces, not the model

Artefact	How it is produced
HTML body	Rendered by application code from the validated sections, using a fixed tag allowlist (<code>p</code> , <code>h2</code> , <code>h3</code> , <code>ul</code> , <code>ol</code> , <code>li</code> , <code>a</code> , <code>strong</code> , <code>em</code> , <code>blockquote</code> , <code>code</code>). Every text node is HTML-escaped unconditionally.
Slug	Derived deterministically from the title, transliterated, lowercased, capped at 72 characters, collisions resolved by numeric suffix. Fixed on first generation and reused on every revision, so a rewrite never changes the URL.
Sources block	Built from the evidence records. Only HTTPS URLs that pass the same validation used for fetching become links.
Tags	Normalised and validated against the entity vocabulary (Section 12).
Word count	Measured on the rendered prose, excluding the Sources block.

Technical note: The React front end renders post bodies with `dangerouslySetInnerHTML` on the assumption that WordPress is a trusted author surface. Building the HTML here, from validated structured data with a fixed allowlist, is the control that keeps that assumption true. A test asserts that model-supplied markup cannot reach WordPress as markup.

10 Editorial Validator

Generation succeeding never means approval. A second, independent pass reviews the draft against the same verified claims the writer was given, and deterministic checks are then applied on top of its verdict.

10.1 What the editorial pass checks

Check	Decided by	Rule as implemented
Unsupported facts	Model	Every factual sentence must map to a supplied claim. Any that does not is a blocking unsupported-claim issue. Numbers, dates, prices, model names and version numbers receive particular attention.
Overstatement	Model	A claim of “faster” rendered as “3× faster”, or “available to some users” rendered as “generally available”, is a blocking overstated-claim issue.
Headline accuracy	Model	If the body does not establish what the headline asserts, that is a blocking misleading-headline issue.
Attribution	Model	Single-source claims must be attributed; conflicting claims must be reported as disagreements or omitted. Otherwise blocking missing-attribution .
Duplicate risk	Model	The draft is compared against the last 100 published titles. Covering the same event is a blocking duplicate issue.
Tone	Model + code	Hype, marketing register and rhetorical questions are hype issues (usually minor). Any of the 15 banned phrases is caught deterministically and blocks.
Structure	Model	Padding and organisational problems are structure issues; grammar is grammar . Usually minor unless pervasive.
Length / format	Code	Word count is compared against the draft’s <i>own</i> format range. Both under and over are advisory , reported distinctly, and never blocking.
Sources block	Code	The rendered HTML must contain a Sources section and the draft must carry at least one source URL. Blocking .
Claim traceability	Code	The draft must cite at least one real claim identifier. Blocking .
Excerpt	Code	Must be present and at least 80 characters. Blocking .
Category	Code	Must be one of the ten allowlisted categories; the writer may refine it only within that closed set.
Tags	Code	More than 5 tags blocks . Fewer than 2 is advisory only — see below.
HTML safety	Code	Any tag outside the renderer’s allowlist blocks .
SEO	Code	Blocking SEO findings are folded in; see Section 11.
Confidence	Code	The model’s own confidence must reach 0.60 .

Why too few tags is advisory but too many blocks

Tags degrade gracefully in the front end, and the tag vocabulary is partly curated, so a story about an unlisted vendor can legitimately end up with a single tag. Blocking on that would withhold a sound article over a metadata nicety that no rewrite could fix. An excessive tag count, by contrast, indicates the model generated tags from article phrasing rather than entities, which is a real defect.

10.2 Decision states

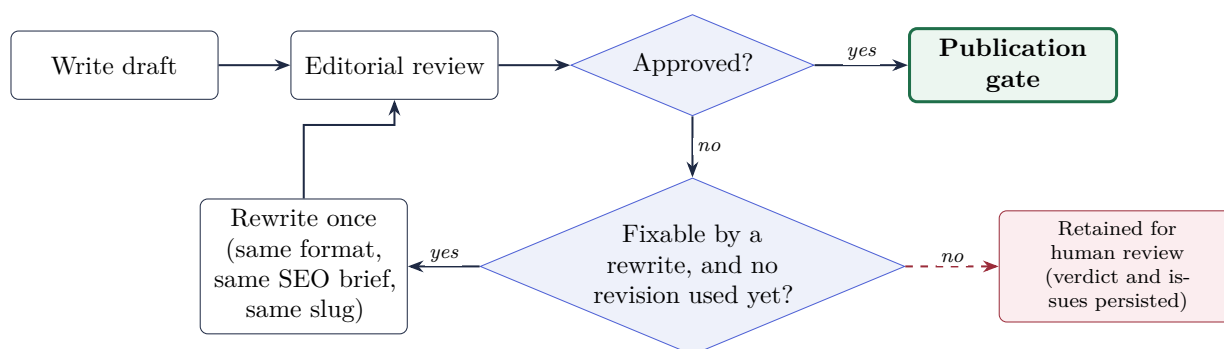
Verdict	Meaning and consequence
approved	No blocking issue from the model, none from the deterministic checks, none from SEO, and confidence ≥ 0.60 . Only this verdict may reach WordPress.
needs-revision	Blocking issues exist but could plausibly be fixed by rewriting from the same claims. One revision may be attempted.
rejected	The draft cannot be fixed from the available claims: it is substantially ungrounded, duplicates a published story, or the material does not support an article.

Code overrides an over-generous model verdict

If the model returns **approved** but any deterministic check found a blocking issue, the verdict is lowered to **needs-revision**. If the model returns **approved** with confidence below **0.60**, the verdict is lowered to **needs-revision**.

If the editorial call **cannot run at all** — provider outage, schema failure, budget exhaustion — the draft is treated as **rejected** with the reason **editorial-review-unavailable**. Failing open here would mean an unreviewed article reaching the CMS, which is precisely what this step exists to prevent.

10.3 The bounded revision loop



Property	Value
Maximum revisions permitted	1 (MAX_REVISION_ATTEMPTS)
Maximum editorial reviews per article	2 (initial, plus one after revision)
When a revision is attempted	Only when the verdict is needs-revision <i>and</i> at least one blocking issue is in the writer-fixable set <i>and</i> the revision allowance is unused.
What a revision may not change	The assigned format, the SEO brief, or the slug. A rewrite fixes issues; it does not renegotiate scope.
What is never writer-fixable	Issues a rewrite cannot resolve — how the renderer emits HTML, how many entity tags the story supports, a missing Sources section. Requesting a rewrite for those would spend two model calls reproducing the same draft.
Outcome if still not approved	The draft is persisted, not discarded , with its verdict, confidence, notes and full issue list, and the story is marked generated with reason editorial:needs-revision or editorial:rejected . Nothing is sent to WordPress.

Technical note: Rejected drafts are retained deliberately: tuning the pipeline is impossible without being able to read what it threw away and why.

11 SEO Judgement Layer

The SEO layer runs *after* verification and *before* writing, so it can shape how an article is found without any power to change what it says.

11.1 The SEO brief

The layer produces one structured object — not free-form advice — precisely so that every field can be checked mechanically afterwards.

Field	Definition and constraints
primaryKeyword	The single query the article should answer. 2–80 characters. Must be grounded in the verified claims.
secondaryKeywords	Up to 6 by schema, then trimmed to the format’s limit (2 for a brief, 4 for standard, 6 for analysis).
searchIntent	One of informational , commercial , navigational , mixed .
seoTitle	Search-tuned headline. 10–80 characters by schema; beyond 60 it is flagged as truncated in results.
metaDescription	50–160 characters, enforced by the schema.
suggestedSlug	A proposal only. Code re-derives the final slug with the same normalisation the article URL uses, so the two can never disagree.
suggestedHeadings	Up to 8 by schema, trimmed to the format’s limit (3 / 6 / 8).
internalLinkTargets	Chosen from a supplied allowlist of real site routes . The model never composes a URL.

Internal links come from a menu, never from invention

A broken internal link is worse than no internal link: a 404 shipped to readers, caused by the step meant to help. The model is handed a list of routes known to exist — `/browse`, `/workflows`, `/compare`, `/new-launches`, `/blog`, plus the slugs of articles *this agent itself published* — and may only choose from it. Anything it returns that is not on the list is silently dropped and recorded in the audit trail. Individual tool and category pages are deliberately absent, because the agent cannot currently confirm those slugs exist.

11.2 Blocking versus advisory

BLOCKING — materially misleading

seo-ungrounded-keyword	The primary keyword is not supported by the verified claims. Sends the wrong readers and misrepresents the article.
seo-misleading-title	The SEO title is not supported by the verified claims — the highest-impact place to overstate, since it is what most readers see.
seo-unsupported-superlative	Any of 24 ranking words (<i>best</i> , <i>top</i> , <i>fastest</i> , <i>leading</i> , <i>#1</i> , <i>revolutionary</i> , <i>industry-leading</i> , ...) appearing in the SEO title, meta description or a keyword unless the same word appears in a verified claim . Such a word asserts a ranking over competitors that were never evaluated.
seo-keyword-stuffing	The primary keyword appearing more than 4 times across title, headings and meta description; or a secondary keyword repeating the primary verbatim.
seo-malformed-slug	The suggested slug does not normalise to a usable URL.
seo-missing-meta-description, seo-missing-primary-keyword	Required metadata absent.

ADVISORY — reported, never blocking

seo-meta-description-short / -long	Outside 50–160 characters.
seo-title-long	Over 60 characters.
seo-keyword-not-in-title / -heading / -intro	Keyword placement is a quality signal, not a correctness one.
seo-heading-not-grounded	A suggested heading has no verified material behind it — headings the article cannot fill are how SEO becomes padding.
seo-too-many-headings, seo-too-many-secondary-keywords	Beyond the format’s shape limits.
seo-intent-mismatch	Commercial intent declared for a brief.

Technical note: “Grounded” is decided by requiring a *majority* of a phrase’s significant tokens to appear in the verified claim set plus the story title. Requiring all of them would reject reasonable paraphrases; requiring one would let “best ai tools 2026” pass on the word “ai” alone. Superlatives are matched on whole-word boundaries, checked deterministically rather than left to a prompt, because this is the single highest-risk way SEO could smuggle an unsupported assertion into a headline.

11.3 Keyword stuffing is measured as repeats, not density

The stuffing check counts occurrences of the primary keyword across the title, the headings and the meta description, with a ceiling of four. It deliberately does not use a density percentage: density targets are what produce robotic copy, and a percentage cannot distinguish a natural second mention from a stuffed one.

11.4 The monotonic decision rule

SEO can lower a verdict; it can never raise one

SEO validation may only *add* issues. It can never clear one, never raise a verdict, and never turn a factually rejected draft into an approved one. Formally, where V is the factual editor’s verdict and B_{seo} the set of blocking SEO findings:

$$V' = \begin{cases} V & \text{if } V \neq \text{approved} \\ \text{needs-revision} & \text{if } V = \text{approved and } |B_{\text{seo}}| > 0 \\ \text{approved} & \text{if } V = \text{approved and } |B_{\text{seo}}| = 0 \end{cases}$$

Search performance is not a reason to publish something untrue. This asymmetry is written as a single dedicated function so that it is testable rather than incidental, and a test asserts that SEO validation cannot rescue a draft the factual editor rejected.

11.5 SEO failure never costs an article

If the SEO call itself fails — timeout, provider outage, budget exhaustion — the brief degrades to *absent* and the writer proceeds without it. An article without a brief ranks less well; an article lost because a metadata call timed out would be a factual story sacrificed to a cosmetic concern. Likewise, if a story has *no verified claims to ground a brief on*, the brief is skipped rather than guessed.

12 Category and Tag Validation

Categories and tags are handled by deliberately different rules, because they are different kinds of thing. A category is a controlled editorial structure; a tag is an open-ended entity vocabulary that grows with the industry.

12.1 Categories: a fixed allowlist

The ten configured categories (internal id / display name)				
ai-models	ai-agents	development	design	image-video
AI Models	AI Agents	Development	Design	Image & Video
productivity	research	mcp	product-updates	industry
Productivity	Research	MCP	Product Updates	Industry

A category is never created while publishing

Two guards apply, in order:

- 1. **Allowlist check, before any network call.** A category outside the ten cannot reach WordPress at all — not as a lookup, and certainly not as a creation. The classifier and the writer both choose from a closed enumeration, and anything unrecognised falls back to industry.
- 2. **Existence check.** If the category is on the allowlist but does not exist in WordPress, the article is **deferred** with reason `category-not-configured` — it is *not* created.

Creating an editorial category as a side effect of publishing one article would mean the public taxonomy grows silently, and it requires a WordPress privilege the agent deliberately does not hold. Seeding categories is an explicit operator action via a separate bootstrap command.

Technical note: Every generated post must carry exactly one real category: the React front end treats a missing category — and WordPress’s default “Uncategorized” — as no category and falls back to a generic label. Tags degrade gracefully in the same interface, so losing one costs discoverability, not correctness. That asymmetry is why category failure defers the post and tag failure does not. Terms are resolved by *slug* and cached for the run; no numeric WordPress identifier is hardcoded anywhere, since identifiers differ per installation.

12.2 Tags: validated, normalised, then created on demand

Every model-proposed tag passes through one validation function before anything downstream may look it up or create it. A tag becomes a permanent public taxonomy term in the CMS, so it is treated as untrusted text reaching a privileged operation.

Rejection reason	Rule
control-characters	Any C0/C1 control character. Checked <i>before</i> tidying, so an embedded newline cannot be collapsed into a space and hidden.
markup	Contains <code><</code> , <code>></code> , or an HTML entity.
empty / too-short	Fewer than 2 characters after tidying.
too-long	More than 40 characters — that is a sentence, not an entity.
too-many-words	More than 3 words. An entity name is at most a few words (“Hugging Face”, “GitHub Copilot”).
disallowed-characters	Outside letters, digits, spaces and the punctuation real product names contain (<code>.</code> <code>+</code> <code>#</code> <code>'</code> <code>&</code> <code>/</code> <code>-</code>).
theme-not-entity	On the blocked list of themes and headline phrasing: “ai”, “machine learning”, “tools”, “news”, “launch”, “best”, “preview”, calendar months, headline verbs, and similar.
unslugifiable	Produces no usable WordPress slug.

Normalisation

- **Canonical forms.** A curated vocabulary of 43 entries collapses spelling variants to one display term — “open ai”, “openai” and “OpenAI” all address the single term *OpenAI*; “copilot” and “github copilot” both become *GitHub Copilot*.
- **Conservative by design.** There is no fuzzy matching anywhere: only exact lookups against a hand-written map and exact word-boundary containment against the article. Two genuinely different products with similar names can never collapse into one.

- **Must be named in the article.** Every accepted tag — including curated-vocabulary hits — must actually appear in the draft. A model proposing “Meta” for a story that never mentions Meta gets nothing, because a wrong tag is a wrong factual association on a published post.
- **Unlisted vendors are still taggable.** A proposal outside the vocabulary is accepted if it behaves like a proper noun the article discusses: capitalised, short, not a theme word, and present in the text. Treating the curated list as exhaustive previously made the tag floor unsatisfiable for any company not on it.
- **Deduplication by slug,** because the slug is what WordPress keys a term on — “OpenAI” and “openai” cannot become two rows in one taxonomy.

Counts and failure behaviour

Rule	Behaviour
Maximum tags	5. Exceeding it is a blocking editorial issue, and the resolver additionally stops at 5 when talking to WordPress.
Minimum tags	2, but advisory only. A post publishes with fewer and a warning is logged.
Tag creation	Governed by <code>WORDPRESS_CREATE_TERMS</code> (default <code>true</code>). Creating a tag requires only <code>edit_posts</code> , which the least-privilege agent account holds.
Tag creation fails	The tag is skipped , never fatal — including on a 403. The article still publishes with whichever tags resolved. A tag is not worth losing an article over; a category is.

13 WordPress Publication Gate

13.1 Draft only — structurally, not by configuration

The agent can only create drafts

Every post the agent creates carries `status = "draft"`. This is **hardcoded in the payload builder**, not derived from configuration:

- The `AGENT_AUTO_PUBLISH` variable is validated to be `false` at startup. If it is set to `true`, the agent **refuses to start** with an explanatory error rather than ignoring the flag.
- That variable is never consulted at the point of publishing, so there is no code path that could honour it even if validation were bypassed.
- The WordPress client asserts the status again at the HTTP boundary and refuses to send any post whose status is not `draft`.
- If WordPress were ever to return a non-draft status, it is logged as an error rather than ignored.

Three tests cover this: that every write a scheduled run makes is a draft, that the client refuses any other status, and that auto-publish cannot be enabled however the variable is spelled.

13.2 Final requirements before a draft is created

#	Requirement	Failure behaviour
1	Editorial approval	Any verdict other than approved skips publication with not-approved . The draft is retained locally.
2	No blocking SEO issue	Already folded into the editorial verdict by the monotonic rule, so this cannot be bypassed.
3	Valid category	Must be on the allowlist <i>and</i> already exist in WordPress. Otherwise deferred: category-not-configured .
4	Safe rendered HTML	The body is built by the renderer from validated structured data with a fixed tag allowlist and unconditional escaping. Any tag outside the allowlist blocks at the editorial stage.
5	Source block present	The rendered content must contain a Sources section and the draft must carry at least one source URL. Blocking.
6	No existing <code>wp_post_id</code>	If this article already has a post, publication is skipped: already-published (wp:n) .
7	Not a duplicate of a published story	If the story was merged into another that already has a post, publication aborts: duplicate-of-published-story .
8	Run draft budget available	If the run has already created <code>AGENT_MAX_DRAFTS_PER_RUN</code> posts, the approved article stays pending and is published by a later run at no model cost.
9	WordPress configured and reachable	If unconfigured, the approved draft is retained (wordpress-not-configured). If unreachable, it is deferred (wordpress-unavailable). Neither loses the article.

13.3 Idempotency: running twice must never produce two drafts

Guard	Mechanism
Before the request	The persisted article row is re-read; if it already carries a <code>wp_post_id</code> , nothing is sent.
Before the request	If the story was merged into another story that already has a post, publication aborts.
After the request	The returned post identifier is written to the database immediately , before the story status update and before the run is finalised. The repository refuses to overwrite an existing identifier.
Across runs	Discovered items are persisted with a unique canonical URL, enforced by a database constraint, so a re-discovered story cannot re-enter generation.

One residual race, stated rather than hidden

The WordPress REST API offers no idempotency key. A crash in the window between “WordPress created the post” and “we stored its identifier” would leave an orphaned draft the next run cannot see. That window is one synchronous database write wide. It is narrowed further by the slug: WordPress derives a unique slug per post, so an orphan surfaces as a -2-suffixed duplicate in the editor rather than as a published duplicate — and the draft-only policy means no reader ever sees either. Closing it fully would require a pre-flight slug lookup on every publish; that trade is revisited if orphans are ever observed. A test covers the crash-after-response case.

13.4 Pending-draft retry

An article can reach **approved**, be persisted, and still fail the WordPress call — the CMS is down, a lookup times out, a proxy is misconfigured. Because its discovered items are already deduplicated, nothing upstream would ever re-offer that story, so without a retry the finished work would sit unpublished forever.

Property	Behaviour
When it runs	First , before ingestion, while the run lock is held.
Cost	One HTTP request per article. Zero model calls — the article is never regenerated, and a test asserts the retry path makes no model call.
Batch size	The smaller of <code>PENDING_PUBLISH.maxPerRun (10)</code> and <code>AGENT_MAX_DRAFTS_PER_RUN</code> .
Abort condition	After 2 consecutive failures the batch stops: WordPress is down, and each further attempt costs a full retry ladder for nothing.
Re-validation	Each candidate is re-read from the database immediately before its attempt, and <code>publishDraft</code> re-applies all three idempotency guards. An article whose editorial status regressed is skipped.
Nothing is discarded	Anything not attempted stays approved with no post identifier and is found by the same query on the next run.
Why first	An authentication failure discovered here disables publishing for the run <i>before</i> the model budget is spent generating posts that could not be delivered.

Authentication failure stops publishing for the whole run

A WordPress 401 or 403 is not retried and disables publishing for the remainder of the run. Every subsequent post would fail identically, and repeatedly hammering an authentication endpoint is how accounts get locked. Approved drafts stay pending and are retried on a later run once an operator has fixed the credentials.

14 Cost and Run Budget Controls

The funnel is designed to be steep: free deterministic filters must do most of the rejecting, so that model calls are spent only on material that has already earned them.

14.1 Implemented caps

Variable	Default	What it bounds
AGENT_MAX_ITEMS_PER_SOURCE_PER_RUN	25	Items taken from any one feed. Overridable per source.
AGENT_MAX_CANDIDATES_PER_RUN	20	Stories sent to the relevance classifier.
AGENT_MAX_STORIES_VERIFIED_PER_RUN	5	Stories taken through evidence gathering, extraction and verification.
AGENT_MAX_ARTICLES_PER_RUN	2	Articles written — the most expensive step.
AGENT_MAX_DRAFTS_PER_RUN	2	Total WordPress posts a run may create, including retries of drafts approved by an earlier run.
AGENT_MAX_LLM_CALLS_PER_RUN	60	Absolute circuit breaker on model calls.
AGENT_MAX_TOKENS_PER_RUN	250,000	Absolute circuit breaker on tokens.

Alongside these, per-task output ceilings bound any single response (classification 512 tokens, extraction 2048, verification 3072, SEO 1024, writing 4096, editing 2048), and task-to-model routing sends cheap work to a fast model and expensive work to a strong one.

14.2 Cap semantics

Value	Meaning
Unset or blank	The documented default applies.
Zero	Zero work — a valid, supported setting. The run defers the remaining work and exits cleanly. Useful for exercising ingestion, deduplication and the pending-publication retry without spending a token.
Anything else invalid (negative, fractional, non-numeric)	Startup fails loudly , naming the variable.

A cost control must fail closed

An earlier implementation silently replaced any unparseable cap with its default. Setting a candidate cap to 0 to mean “score nothing” therefore produced the *full default* workload — a cost control failing *open* at the exact moment an operator was tightening it. The current parser accepts zero as meaningful and rejects everything else loudly. A mistyped 0 now yields a no-op run, never a full one, and three tests cover this behaviour.

14.3 How work is deferred when a cap is reached

Cap reached	Recorded deferral reason
Candidate cap, before classification	candidate-budget-reached
Verification cap	verification-budget-reached
Article cap	article-budget-reached
Model call or token budget	llm-budget-exhausted
Draft cap, after approval	The approved article stays pending with no post identifier and is published by the next run at no model cost.

Deferring is not rejecting

Deferred stories are re-queued on the next run **ahead of** newly discovered candidates, because they already carry a score. They are not resurrected blindly: each one is re-checked against the staleness gate using the current clock, so a deferral that aged past 96 hours is then rejected rather than lingering indefinitely. Two tests cover both halves of this behaviour.

14.4 Two things a cap never stops

- **Publishing work an earlier run already approved.** Posting a finished draft costs no model budget, so zeroing every model cap still drains the pending queue. This is deliberate, and a test asserts it. To stop all CMS writes, set `AGENT_MAX_DRAFTS_PER_RUN` to 0.
- **The kill switch.** `AGENT_ENABLED` defaults to `true`, so an unset variable never silently pauses a production schedule. Set to `false`, the process exits cleanly *before* opening the database, resolving the model provider, or touching the network.

14.5 Concurrency

One invocation is one complete run; there is no daemon and no timer. A database run lock prevents overlapping executions — a second run that starts while one is active exits cleanly having done nothing. A lock older than `AGENT_RUN_LOCK_STALE_MINUTES` (default 30) is reclaimed, so a crashed run cannot block the schedule permanently.

15 Approval Decision Flow

The diagram below traces one story from discovery to a WordPress draft. The centre column is the path a successful story takes. Branches to the right are **terminal rejections**; branches to the left are **deferrals**, where the story survives and is reconsidered on the next run.

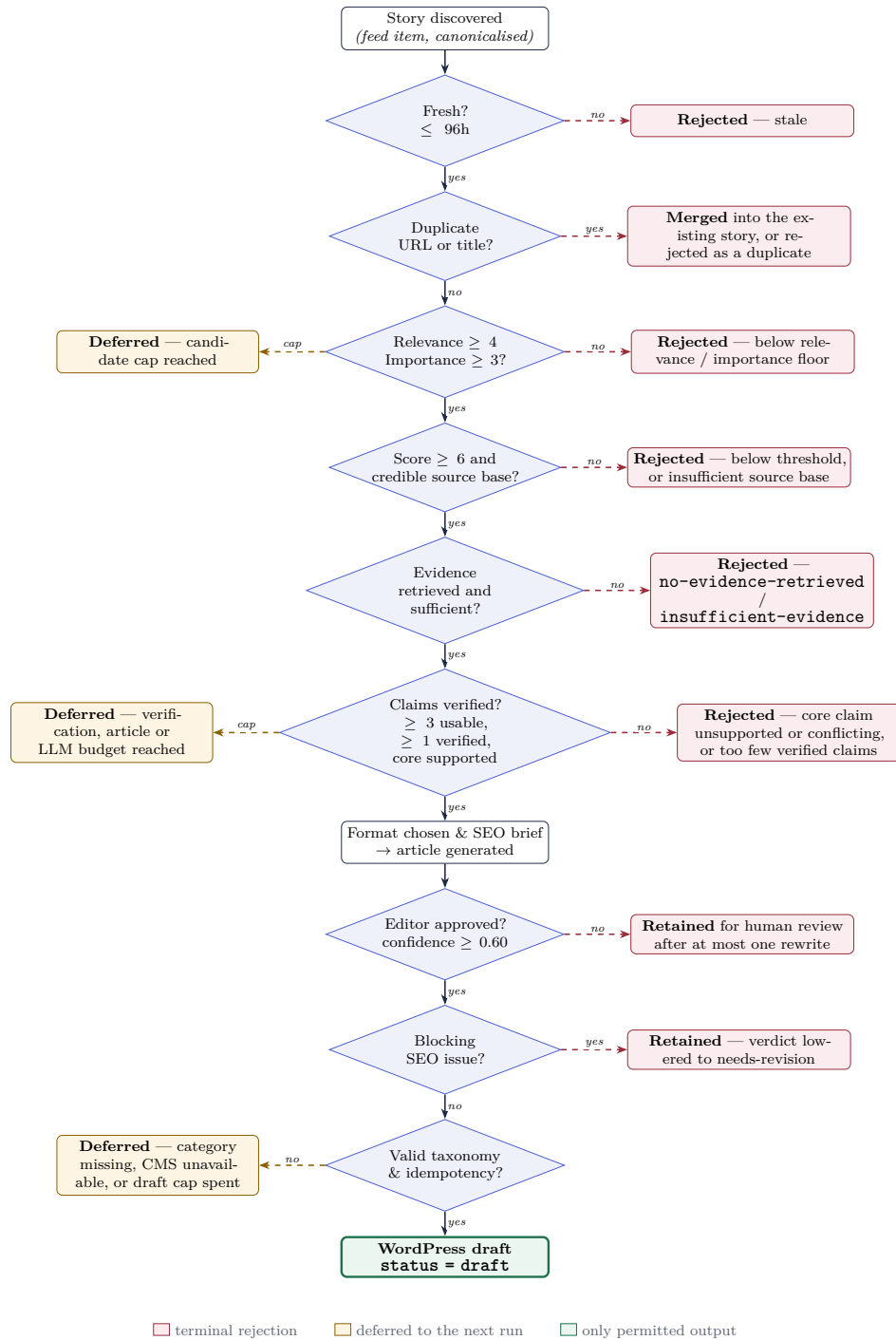


Figure 1: End-to-end approval flow. Every diamond is a gate that must be cleared; there is no path to the green terminal that bypasses one.

16 Decision Matrix

The complete set of gates, in pipeline order. **Blocking** gates prevent publication of that story; **deferral** gates pause it without judging it.

#	Stage / gate	Pass condition	Failure outcome
BLOCKING GATES			
1	Automation enabled	AGENT_ENABLED is true	Run exits 0 having done nothing
2	Run lock	No other run is active	Run exits 0; note records the holder
3	Source enabled	enabled: true in the registry	Source is not read at all
4	Banned domain	Item host is not among the 9 banned domains	Dropped at ingestion; banned-domain
5	Title is a real headline	≥ 12 characters	Dropped; title-too-short
6	URL deduplication	Canonical URL not already persisted	Item dropped as a duplicate
7	Story not terminal	Status is not published or rejected, and it is not a merged duplicate	already-published, previously-rejected, duplicate-of-existing-story
8	Freshness	Freshest item ≤ 96 hours old	Rejected; stale (nh old)
9	Hard-rejected topic	No hard-reject keyword rule matches	Rejected; excluded-topic: id
10	Has source items	At least one item backs the story	Rejected; no-source-items
11	Relevance floor	Classifier relevance ≥ 4	Rejected; below-relevance-floor
12	Importance floor	Classifier importance ≥ 3	Rejected; below-importance-floor
13	Weighted threshold	$S \geq 6.0$	Rejected; below-threshold
14	Credible source base	Tier 1 present, or ≥ 2 independent publishers	Rejected; insufficient-source-base
15	Evidence sufficiency	≥ 1 Tier 1 document retrieved, or ≥ 2 distinct Tier 2 publishers	Rejected; no-evidence-retrieved or insufficient-evidence
16	Source integrity	No retrieved document attempts prompt injection	Document quarantined; story rejected if sufficiency is lost
17	Claim extraction	At least one atomic claim extracted	Rejected; no-claims-extracted
18	Core claim support	Core claim is not unsupported, and not conflicting without Tier 1 resolution	Rejected; core-claim-unsupported / core-claim-conflicting
19	Verified-claim floor	≥ 3 usable claims and ≥ 1 fully verified	Rejected; too-few-verified-claims / no-fully-verified-claims
20	Hard word ceiling	Rendered draft ≤ 1400 words	Draft rejected as runaway output
21	Deterministic editorial checks	Sources section present, source URLs recorded, claim traceability, excerpt ≥ 80 chars, ≤ 5 tags, no banned phrase, only allowlisted HTML tags	Verdict forced to needs-revision; draft retained, never sent

#	Stage / gate	Pass condition	Failure outcome
22	Editorial verdict	Model returns approved with no blocking issue	Retained; editorial:needs-revision or editorial:rejected
23	Confidence floor	Editorial confidence ≥ 0.60	Verdict lowered to needs-revision; low-confidence
24	SEO blocking findings	No blocking SEO issue	Approved verdict lowered to needs-revision
25	Publication idempotency	No existing wp_post_id ; story is not a duplicate of a published one	Skipped; already-published or duplicate-of-published-story

DEFERRAL GATES — the story survives and is reconsidered next run

D1	Candidate cap	Fewer than 20 stories classified this run	candidate-budget-reached
D2	Verification cap	Fewer than 5 stories verified this run	verification-budget-reached
D3	Article cap	Fewer than 2 articles written this run	article-budget-reached
D4	Model budget	Call and token budgets not exhausted	llm-budget-exhausted
D5	Draft cap	Fewer than 2 posts created this run	Approved article stays pending
D6	Category exists in CMS	Category is present in WordPress	category-not-configured
D7	CMS reachable	WordPress responds	wordpress-unavailable / wordpress-not-configured

Category	Count	Effect
Blocking gates	25	Failure prevents publication of that story
Deferral gates	7	Failure pauses the story without judging it

17 Examples

The three walkthroughs below describe how the implemented pipeline behaves in each of its three outcomes. Story details are illustrative; every threshold, reason code and behaviour quoted is the real one.

17.1 Example A — Accepted

Stage	What happens
Discovery	A vendor's official blog (Tier 1) and two technology publications (Tier 2) each publish on the same capability change. Four items arrive across three feeds.
Deduplication	Canonical URLs differ, so no item is dropped at level 1/2. Title similarity is above 0.62 and the headlines share two distinctive tokens, so all four items cluster into one story from three independent publishers.
Freshness & rules	Freshest item is 5 hours old (within 96). No banned domain, no hard-rejected topic. Included-topic rules match model-update and capability ; two audience entities are named.
Scoring	Classifier returns $R = 8$, $I = 7$. Trust: best tier 1 gives 10, plus 2 corroboration points, capped at 10. Freshness at 5 hours ≈ 9.5 . $S_{\text{base}} = 0.35(8) + 0.30(7) + 0.20(10) + 0.15(9.5) = 8.33$; the topic prior adds the clamped maximum +2, so $S = 10.0$ after clamping.
Selection	Clears all four gates: $R \geq 4$, $I \geq 3$, $S \geq 6$, Tier 1 present. Selected , and sorted first by score.
Evidence	Tier 1 fetched first, then the two Tier 2 pages; the fourth is beyond the 4-page budget. Three documents retrieved, all above the 200-character floor, none flagged for injection. Tier 1 present $\Rightarrow$ sufficient .
Claims	Extraction runs once per document. Overlapping claims merge, leaving 14 distinct claims across types launch , capability , pricing and date .
Verification	The core launch claim is confirmed by Tier 1 and both Tier 2 publishers $\Rightarrow$ verified . capability and pricing claims are backed by Tier 1 $\Rightarrow$ verified . Two claims appear only in one publication $\Rightarrow$ single-source . Result: 9 verified , 2 single-source . Passes all four story-level gates.
Format	9 verified claims, 3 sources, 3 publishers $\Rightarrow$ meets standard but not analysis (needs 12 claims and importance ≥ 7 — importance qualifies, claim count does not). Format: standard, 500–900 words .
SEO brief	Grounded primary keyword, 4 secondary keywords, a 148-character meta description, a normalised slug, and one internal link chosen from the route allowlist.
Writing	11 usable claims supplied; the 3 unsupported ones are withheld entirely. The two single-source claims are attributed in the prose. Draft comes in at 690 words across 6 sections.
Editorial	No unsupported statements, headline matches the body, attribution present, no banned phrase, 4 entity tags, only allowlisted HTML, Sources section rendered. Confidence 0.82 ≥ 0.60 . Verdict: approved .
SEO validation	No blocking findings; one advisory (seo-keyword-not-in-heading). Verdict unchanged.
Publication	Category on the allowlist and present in WordPress; 4 tags resolved (one created); no existing post identifier; draft budget available.
Outcome	WordPress draft created, status = draft , awaiting human review.

17.2 Example B — Rejected

Stage	What happens
Discovery	A single Tier 1 vendor feed carries a strong, specific headline about a major capability change. One item, one publisher.
Freshness & rules	3 hours old. No banned domain, no hard-rejected topic. Included-topic rules match strongly.
Scoring	Classifier returns $R = 9$, $I = 8$ from the headline and feed summary. Trust = 10 (Tier 1, no corroboration bonus). Freshness ≈ 9.7 . $S = 9.1$ — comfortably above the threshold.
Selection	Passes: R and I clear their floors, $S \geq 6$, and the source-base gate is satisfied because a Tier 1 source is present. Selected — at this point the story looks excellent.
Evidence	The agent fetches the article URL. The publisher returns HTTP 403 . As a 4xx it is not retried ; the fetch is counted as failed and contributes no evidence. No other item backs the story, so there is nothing else to fetch.
Sufficiency	Zero documents retrieved. $\Rightarrow$ insufficient .
Outcome	Rejected , reason no-evidence-retrieved . The story is marked with evidence state insufficient , the reason is persisted, and no model call is made beyond the classifier .

Why this is the correct outcome

The agent had a trusted publisher, a high score, and a headline specific enough to write from. It still produced nothing, because it could not read the document. A system willing to write from a headline would have produced a confident, plausible, *unverified* article here — which is exactly the failure this pipeline is built to make impossible. Had a second independent publisher also covered the story and been reachable, evidence sufficiency would have been met by two Tier 2 publishers and the story would have continued, with capability and pricing claims downgraded to **single-source** and attributed.

17.3 Example C — Deferred

Stage	What happens
Context	A busy run. Two higher-scoring stories have already been written, and <code>AGENT_MAX_ARTICLES_PER_RUN</code> is at its default of 2 .
This story	A legitimate, well-sourced launch scoring $S = 7.4$: Tier 1 present, fresh, relevant. It clears every selection gate and is selected , but sorts third by weighted score.
The cap	On reaching it, the loop finds <code>articlesGenerated (2) ≥ maxArticles (2)</code> .
Action taken	The story is set to deferred with reason <code>article-budget-reached</code> . The run's deferred counter increments. No evidence is gathered, no claims extracted, no model budget spent on it.
Next run	Deferred stories are loaded <i>first</i> and queued ahead of newly discovered candidates, because they already carry a score. This story is re-checked by the prefilter against the current clock.
If still fresh	It proceeds through evidence, verification, writing and review exactly as any other story would.
If now stale	Having aged past 96 hours, it is rejected with stale (nh old) rather than lingering as a permanent backlog entry.
Outcome	Deferred — a statement about the run's budget, not about the story's quality.

The same behaviour at every cap

Identical logic applies at each of the five budget boundaries. The candidate cap defers before classification; the verification cap defers before evidence gathering; the article cap defers before writing; model call and token budgets defer at whatever point they are reached; and the draft cap leaves an *already-approved* article pending, to be posted by the next run at no model cost. In every case the reason is recorded and the story keeps its place.

18 What the LLM Does vs What Code Decides

This division is the core of the system's trustworthiness. The model contributes judgement; code contributes authority.

Decided by the model	And how code constrains it
Semantic relevance and importance (0–10)	Clamped to range, combined with code-computed trust and freshness under fixed weights, then checked against independent floors the model cannot see or influence.
Novelty score	Not consumed. Produced and validated, read by nothing.
Editorial category	Constrained to a closed enumeration of 10; anything unrecognised falls back to industry .
Recommendation (proceed/skip)	Advisory only. Logged and displayed; never decisive.
Claim extraction	Bounded to 25 claims per source, one source per call, closed type enumeration, strict schema. Instruction-shaped claims are dropped by code. Support level is not set here.
Claim verification	Supporting URLs are filtered to the story’s real evidence set; tier rules then downgrade any verdict the evidence does not justify; ignored claims default to unsupported. The rules never upgrade.
Article prose	Receives only verified and single-source claims — never source documents. Returns structured sections, never markup. Hard word ceiling enforced by code.
Editorial review	Its approved verdict is overridden by any deterministic blocking issue and by a confidence below 0.60. If the review cannot run, the draft is rejected, not waved through.
SEO brief	Every field is re-checked against the verified claims by code. The slug is re-derived. Internal links are filtered against a route allowlist.

Decided by deterministic code	Implementation
Source enablement and tiering	Static registry; the model never sees a feed list or chooses a source.
Freshness	Arithmetic on timestamps against a 96-hour window.
Duplicate detection	URL canonicalisation, database uniqueness constraint, and token/bigram title similarity with fixed thresholds.
Threshold enforcement	Relevance, importance, weighted score and source-base gates — all applied after the model has answered, to values it cannot revise.
Cost caps	Call and token budgets, per-stage caps, enforced by a circuit breaker that throws before a call is made.
Article format and length band	Computed from verified-claim and publisher counts. Explicitly not a model call, so the model cannot argue for a longer article.
Category allowlist	Ten fixed terms; never created during publishing.
Tag validation and count	Character class, length, word count, markup, control characters, theme-vs-entity shape, presence in the article, slug deduplication, maximum of 5.
HTML safety	Fixed tag allowlist, unconditional escaping, validated link targets. The model never supplies markup.
Slug generation	Deterministic from the title; stable across revisions and retries.
Draft-only policy	Hardcoded in the payload, asserted again at the HTTP boundary, with startup refusing an auto-publish configuration.
Idempotency	Three publication guards plus an immediate, non-overwritable database write of the post identifier.
Network safety	HTTPS-only, SSRF address checks, response size caps, timeouts, bounded redirects.
Secret handling	Credentials registered for redaction; a test asserts no credential appears in logs even at debug level.

The model has no publishing authority

At no point does a model decide that something is published. It cannot enable a source, raise its own score past a threshold, extend its own word budget, create a category, emit markup, set a URL, choose a post status, or call the CMS. Its most favourable possible output is a *recommendation* that deterministic code then accepts or refuses. Conversely, a model verdict of “reject” is always honoured: the asymmetry runs one way only.

19 Limitations

Stated plainly, because a system whose limits are known is easier to operate than one whose limits are discovered.

19.1 Source access

- **Blocked source URLs.** Some publishers return HTTP 403 to automated readers. Those documents contribute no evidence and, if the story depends on them, it is rejected. This is a correct outcome but a real coverage loss.
- **Paywalls, consent walls and JavaScript shells.** A page yielding under 200 characters of extractable text is discarded. No headless browser is used, deliberately.

- **Six Tier 1 vendors are currently unreachable.** Anthropic, Meta AI, Mistral, Microsoft, Runway — and Ars Technica at Tier 2 — publish no stable feed the agent can use. They are registered as disabled with the reason recorded, rather than pointed at a guessed URL that would fail silently. News from those vendors is therefore only picked up when a configured Tier 2 publication covers it.
- **No Tier 3 sources are configured.** Tier 3 exists in the trust model and is the default assigned to unrecognised hosts, but nothing currently feeds it, so its behaviour is exercised only by tests.
- **Article-text extraction is heuristic.** Boilerplate removal is pattern-based; a wrong guess degrades text quality rather than creating a safety issue, since the output is plain text either way.
- **Publisher independence is approximated** by registrable domain, without a public-suffix list. Occasional mistakes on multi-part country domains are possible and harmless.

19.2 Model and API availability

- A provider outage, an exhausted credit balance or a rejected key stops the run rather than degrading quality — an invalid credential fails identically for every story, so it aborts instead of rejecting the whole queue one call at a time.
- Structured-output failures get one repair pass and one clean retry (3 attempts). Persistently invalid output is rejected, never accepted.
- A model refusal surfaces as a refusal, never as article content.
- The editorial pass failing means *rejection*, not approval. Availability problems therefore cost articles, never quality.

19.3 Evidence and coverage

- At most 4 evidence pages per story, and no links are followed out of fetched pages. The agent cannot go looking for corroboration that its own feeds did not surface.
- A legitimate article *about* prompt injection will be quarantined by the injection detector. This false positive is accepted deliberately: losing a meta-story is cheaper than republishing manipulated content, and the rejection is logged for a human to overturn.
- Deduplication may occasionally merge two genuinely distinct stories. The bias toward merging is deliberate — a wrong merge loses one article, a wrong split publishes a duplicate.

19.4 Article shape

- **Thin stories produce short briefs**, by design. A 180–350 word article where every sentence is grounded is a success, not a shortfall. Clients reviewing output should expect briefs to be common.
- The `analysis` format requires 12 verified claims, 3 sources, 3 publishers *and* importance ≥ 7 , so it will be rare.
- The optional editorial “AI Tool Kart take” section described in the design brief is not implemented; the writer is given five factual headings only.

19.5 SEO metadata

The SEO brief is stored locally, not written to the CMS

The production site runs All in One SEO, which was inspected before this layer was built. It offers no safe write path for the agent: its fields appear on reads but are absent from the writable post schema, its own save route is an undocumented internal endpoint, and the agent's least-privilege role receives HTTP 403 from its options endpoint. Writing SEO fields would mean reverse-engineering an undocumented endpoint *and* raising the agent's CMS privileges — both poor trades for metadata a human can set in seconds during review.

So the brief is persisted with the article and surfaced to the reviewer, and the CMS sink is a deliberate no-op behind a one-file interface. What does still reach the site is the title and the excerpt — which matters, because the plugin's stock description template is the post excerpt, so a well-written excerpt commonly becomes the rendered meta description with no plugin API at all.

19.6 Featured images

No featured image is ever attached. The field is absent from the post payload entirely, and a test asserts it. The front end already renders a gradient fallback in the same frame when an image is missing, so the grid stays on its baseline. Automatic image generation is an explicit non-goal at this stage.

19.7 Human review remains required

Nothing reaches a reader without a person

The agent's output is a WordPress draft. Publication is a human action. Every gate in this document reduces the volume and raises the floor of what a reviewer sees; none of them replaces the reviewer. The draft carries everything needed to check it without leaving the editor: the rendered Sources block, the claim identifiers used, the editorial verdict, confidence score and full issue list, the assigned format and its target range, and the SEO brief.

20 Conclusion

Whether a story becomes an article is not a single judgement. It is the product of eight independent layers, each of which can stop the story on its own:

Layer	Contribution
Deterministic filters	Source enablement, banned domains, title validity, freshness, URL and title deduplication, hard-rejected topics — all free, all before any model call.
Weighted scoring	Four dimensions under fixed weights, with independent relevance, importance, score and source-base floors.
Evidence quality	Documents must be <i>retrieved</i> , not merely referenced, and must meet a tier-based sufficiency rule.
Fact verification	Per-claim support levels, with deterministic tier rules that only ever downgrade, and four story-level gates.
Editorial review	An independent pass against the same claims, with code overriding an over-generous verdict and a bounded single revision.
SEO review	Grounding, superlative and stuffing checks that can lower a verdict but never raise one.
Taxonomy validation	A fixed category allowlist never extended at publish time, and a validated entity-tag vocabulary.
Budget and safety controls	Per-run caps that defer rather than reject, HTML and URL safety boundaries, idempotency guards, and a draft-only publishing policy.

The design preference, stated once more

no article is always preferred to unsupported article

Every ambiguous case in this pipeline resolves toward not publishing. Evidence that cannot be retrieved ends the story. A claim the sources do not establish is removed rather than softened. A review that cannot run is a rejection, not a pass. A budget that runs out pauses work rather than lowering the bar. And the most favourable outcome the system can produce on its own is a draft, waiting for a person to decide.

This behaviour is not merely intended; it is asserted. The agent’s test suite covers the selection thresholds, the tier downgrade rules, the format decision, the dedupe thresholds, the revision bound, the SEO monotonicity rule, the tag and category validation, the HTML allowlist, the injection quarantine, the draft-only policy, the idempotency guards, the cap semantics including the fail-closed zero case, and the pending-publication retry. All 422 tests pass at the time of writing.

Appendix A. Current Configurable Parameters

Every parameter below materially affects story selection, article approval, or publication. Environment variables are shown with their documented defaults; code constants are shown with their current values and the file that owns them.

No credentials appear in this document

LLM_API_KEY, WORDPRESS_USERNAME and WORDPRESS_APP_PASSWORD are listed by *name and purpose only*. Their values are never printed here, never logged by the agent (secrets are registered for redaction at startup, and a test asserts no credential appears even at debug level), and never exposed to the browser bundle.

A.1 Environment variables (25)

Variable	Purpose	Default
LLM_PROVIDER	Which model provider to use; mock runs the pipeline offline with a deterministic provider	mock
LLM_API_KEY	Provider credential. Required for any non-mock provider; the agent refuses to start without one	<i>(not set)</i>
LLM_MODEL_FAST	Override for the cheap model class (classification, extraction, SEO)	adapter default
LLM_MODEL_STRONG	Override for the strong model class (verification, writing, editing)	adapter default
WORDPRESS_API_URL	CMS REST base including namespace. HTTPS required except for loop-back/.local development hosts	<i>(not set)</i>
WORDPRESS_USERNAME	Dedicated least-privilege CMS account (not an administrator)	<i>(not set)</i>
WORDPRESS_APP_PASSWORD	CMS application password	<i>(not set)</i>
WORDPRESS_CREATE_TERMS	Whether missing tags may be created. Categories are never created regardless	true
AGENT_DB_PATH	SQLite database path (dedupe, idempotency, audit trail)	./data/news-agent.sqlite
AGENT_ENABLED	Automation kill switch. False exits cleanly before the database, provider or network are touched	true
AGENT_AUTO_PUBLISH	Must be false. True causes startup to fail ; no code path could honour it	false
AGENT_MAX_ITEMS_PER_SOURCE_PER_RUN	Items taken from any one feed per run	25
AGENT_MAX_CANDIDATES_PER_RUN	Stories sent to the relevance classifier	20
AGENT_MAX_STORIES_VERIFIED_PER_RUN	Stories taken through evidence, extraction and verification	5
AGENT_MAX_ARTICLES_PER_RUN	Articles written per run	2
AGENT_MAX_DRAFTS_PER_RUN	Total CMS posts per run, retries included	2
AGENT_MAX_LLM_CALLS_PER_RUN	Absolute circuit breaker on model calls	60
AGENT_MAX_TOKENS_PER_RUN	Absolute circuit breaker on tokens	250000
AGENT_MIN_WEIGHTED_SCORE	Weighted score (0–10) a story must reach to earn evidence gathering	6
AGENT_HTTP_TIMEOUT_MS	Per-request timeout for feeds and evidence	15000

Variable	Purpose	Default
AGENT_MAX_RESPONSE_BYTES	Response size cap	2500000
AGENT_USER_AGENT	Identifying user agent sent to publishers	AIToolKart NewsAgent/0.1
AGENT_RUN_LOCK_STALE_MINUTES	Age at which a run lock is reclaimed	30
AGENT_LOG_FORMAT	<code>json</code> for production, <code>pretty</code> for humans	<code>pretty</code>
AGENT_LOG_LEVEL	Log verbosity	<code>info</code>

A.2 Scoring and selection constants (12)

Constant	Value	Purpose
SCORE_WEIGHTS.relevance	0.35	Weight on classifier relevance
SCORE_WEIGHTS.importance	0.30	Weight on classifier importance
SCORE_WEIGHTS.sourceTrust	0.20	Weight on computed source trust
SCORE_WEIGHTS.freshness	0.15	Weight on computed freshness
TIER_TRUST_SCORE	10 / 7 / 3	Trust contributed by Tier 1 / 2 / 3
CORROBORATION_BONUS	1	Points per additional independent publisher
MAX_CORROBORATION_BONUS	2	Ceiling, so rewrites cannot outrank a primary source
FRESHNESS_HALF_LIFE_HOURS	48	Freshness decays linearly to zero over twice this
MIN_RELEVANCE	4	Hard relevance floor, independent of the score
MIN_IMPORTANCE	3	Hard importance floor
Topic-prior clamp	± 2	Limit on how far keyword rules may move a score
Entity-bonus cap	3×0.5	Limit on audience-entity contribution

Technical note: `src/config/limits.ts`, `src/ranking/score.ts`, `src/ranking/prefilter.ts`.

A.3 Deduplication and freshness (6)

Constant	Value	Purpose
DEDUPE.sameStoryThreshold	0.62	Title similarity at or above this is one story
DEDUPE.ambiguousThreshold	0.50	Merge but flag; same-publisher guard applies
DEDUPE.clusterWindowHours	72	Titles only cluster within this window
maxStoryAgeHours	96	Staleness gate, measured from the freshest item
minTitleLength	12	Below this a title is a stub, not a story
MAX_SLUG_LENGTH	72	Slug ceiling, truncated on a word boundary

A.4 Evidence and verification (8)

Constant	Value	Purpose
MAX_EVIDENCE_FETCHES_PER_STORY	4	Evidence pages fetched per story
MAX_EVIDENCE_CHARS	24,000	Cleaned text retained per source
Minimum extracted text	200 chars	Below this a page is a paywall or JS shell
MAX_CLAIMS_SENT	25	Claims sent to the verifier per story
EVIDENCE_EXCERPT_CHARS	6,000	Evidence excerpt length in the verifier prompt
MIN_VERIFIED_CLAIMS	3	Usable claims required before writing
CLAIM_RULES	8 typed rules	Per-claim-type tier requirements (Section 7)
DEFAULT_CLAIM_RULE	1 Tier 2	Fallback for an unrecognised claim type

A.5 Article and editorial constraints (15)

Constant	Value	Purpose
ARTICLE_FORMATS.brief	180–350 words, 3–5 sections	Floor format
ARTICLE_FORMATS.standard	500–900 words, 4–7 sections	Common case
ARTICLE_FORMATS.analysis	900–1400 words, 5–9 sections	Only when evidence carries it
DEFAULT_ARTICLE_FORMAT	standard	Fallback for pre-format records
FORMAT_THRESHOLDS.standard	6 claims / 2 sources / 2 publishers	Bar for standard
FORMAT_THRESHOLDS.analysis	12 claims / 3 sources / 3 publishers / importance 7	Bar for analysis
ARTICLE.hardMaxWords	1400	Runaway-output rejection ceiling
ARTICLE.minTags	2	Advisory tag floor
ARTICLE.maxTags	5	Blocking tag ceiling
ARTICLE.excerptMinChars	80	Blocking excerpt floor
ARTICLE.excerptMaxChars	320	Excerpt ceiling
ARTICLE.titleMaxChars	110	Title ceiling
MAX_REVISION_ATTEMPTS	1	Bounded writer/editor loop
MIN_APPROVAL_CONFIDENCE	0.60	Editorial confidence floor for approval
ALLOWED_TAGS	11 HTML tags	Renderer allowlist: p, h2, h3, ul, ol, li, a, strong, em, blockquote, code

A.6 SEO constraints (11)

Constant	Value	Purpose
SEO.metaDescriptionMinChars	50	Below this is not a description
SEO.metaDescriptionMaxChars	160	Truncation point in search results
SEO.seoTitleMaxChars	60	Soft target; beyond it, advisory warning
SEO.seoTitleHardMaxChars	80	Schema ceiling
SEO.maxSecondaryKeywords	6	Schema ceiling
SEO.maxSuggestedHeadings	8	Schema ceiling
SEO.maxInternalLinks	4	Schema ceiling
SEO.maxPrimaryKeywordRepeats	4	Stuffing threshold (a count, not a density)
SEO_BY_FORMAT	3 / 6 / 8 headings; 2 / 4 / 6 keywords; 2 / 3 / 4 links	Per-format shape, so SEO cannot become padding
UNSUPPORTED_SUPERLATIVES	24 terms	Ranking words blocked unless verified
STATIC_ROUTES	5 routes	Internal-link allowlist, plus this agent's own published slugs

A.7 Taxonomy and tags (6)

Constant	Value	Purpose
EDITORIAL_CATEGORIES	10 terms	Fixed category allowlist; never extended at publish time
TAG_ENTITIES	43 entries	Curated entity vocabulary and canonical display forms
NON_ENTITY_TAGS	60+ terms	Theme words, months and headline verbs that may never be tags
MAX_TAG_CHARS	40	Longer is a sentence, not an entity
MIN_TAG_CHARS	2	Minimum usable tag
MAX_TAG_WORDS	3	An entity name is at most a few words

A.8 Editorial scope lists (4)

Constant	Value	Purpose
INCLUDED_TOPICS	10 rules, +1 to +2.5	Positive topic prior
EXCLUDED_TOPICS	8 rules; 5 hard-reject	Noise suppression and hard vetoes
BANNED_DOMAINS	9 hosts	Aggregators and content mills, dropped at ingestion
MAX_SUMMARY_CHARS	1,200	Feed-summary truncation (classifier context only)

A.9 Retry and operational bounds (10)

Constant	Value	Purpose
RETRY.httpAttempts	3	Transient HTTP failures only; 4xx is never retried
RETRY.httpBaseDelayMs	400	Backoff base
RETRY.httpMaxDelayMs	4,000	Backoff ceiling
RETRY.llmSchemaAttempts	3	One repair pass, then one clean retry
RETRY.wordpressAttempts	3	CMS 5xx retries; 401/403 are never retried
RETRY.maxRedirects	3	Redirects followed, each revalidated
PENDING_PUBLISH.maxPerRun	10	Backlog drained per run
PENDING_PUBLISH.maxConsecutiveFailures	2	Batch abort when the CMS is down
TASK_MODEL_CLASS	6 task mappings	fast: classify, extract, SEO; strong: verify, write, edit
TASK_MAX_OUTPUT_TOKENS	512 / 2048 / 3072 / 1024 / 4096 / 2048	Per-task output ceilings

Section	Parameters
A.1 Environment variables	25
A.2 Scoring and selection constants	12
A.3 Deduplication and freshness	6
A.4 Evidence and verification	8
A.5 Article and editorial constraints	15
A.6 SEO constraints	11
A.7 Taxonomy and tags	6
A.8 Editorial scope lists	4
A.9 Retry and operational bounds	10
Total documented	97

Appendix B. Implemented Rejection / Deferral Reasons

Every outcome the agent records carries one of the machine-readable reasons below. These are the actual strings written to the database and the run log — nothing here is illustrative. Where a reason interpolates a measured value, the pattern is shown in *italics*.

B.1 Rejections — discovery and eligibility

Reason code	Meaning
already-published	The story was already carried to publication
previously-rejected	The story was already rejected; the original reason is retained
duplicate-of-existing-story	Merged into another story record
no-source-items	No news item backs the story
banned-domain	Every backing item is from a banned host
title-too-short	Headline below 12 characters
stale (<i>n</i> h old)	Freshest item older than 96 hours
excluded-topic: <i>rule-id</i>	A hard-reject topic rule matched — one of stock , doom , clickbait , listicle , horoscope-noise

B.2 Rejections — scoring and selection

Reason code	Meaning
below-relevance-floor (<i>r</i> < 4)	Classifier relevance under the hard floor
below-importance-floor (<i>i</i> < 3)	Classifier importance under the hard floor
below-threshold (<i>s</i> < 6)	Weighted score under the configured threshold
insufficient-source-base (no Tier 1, <i>n</i> independent Tier 2)	Neither a Tier 1 source nor two independent publishers
classification-failed	The relevance call failed for this story specifically

B.3 Rejections — evidence, claims and verification

Reason code	Meaning
no-evidence-retrieved	Not one source document could be read
insufficient-evidence (no Tier 1, <i>n</i> independent Tier 2)	Evidence was retrieved but does not meet the tier rule
(<i>the above</i>) + (<i>n</i> source(s) quarantined for prompt injection)	Appended when documents were withheld for attempted injection
no-claims-extracted	No atomic factual claim could be extracted
core-claim-unsupported	The claim the story rests on is not supported by the evidence
core-claim-conflicting	Sources disagree on the core claim and no Tier 1 source resolves it
too-few-verified-claims (<i>n</i> < 3)	Fewer than three usable claims survived
no-fully-verified-claims	Every surviving claim is single-source only

B.4 Editorial and SEO — draft retained, not published

Reason / issue code	Meaning
<code>editorial:needs-revision</code>	Story status after an unapproved draft that a rewrite might fix
<code>editorial:rejected</code>	Story status after an unfixable draft
<code>editorial-review-unavailable</code>	The editorial call could not run; treated as rejection
<code>low-confidence ($c < 0.6$)</code>	Approval withheld on the confidence floor
<i>Blocking deterministic editorial issues</i>	
<code>too-many-tags ($n > 5$)</code>	Tag ceiling exceeded
<code>excerpt-too-short</code>	Excerpt under 80 characters
<code>no-source-urls</code>	The draft records no source URL
<code>missing-sources-section</code>	The rendered body has no Sources block
<code>no-claim-traceability</code>	The draft cites no claim identifier
<code>banned-phrase: "phrase"</code>	One of the 15 banned editorial phrases appears
<code>disallowed-html-tags: tags</code>	Markup outside the renderer allowlist
<i>Model-raised blocking issue kinds</i>	
<code>unsupported-claim,</code> <code>overstated-claim,</code> <code>misleading-headline,</code> <code>missing-attribution, duplicate</code>	Factual and attribution failures
<code>hype, structure, grammar</code>	Usually minor unless pervasive
<i>Advisory only — reported, never blocking</i>	
<code>too-few-tags ($n < 2$)</code>	Below the tag target
<code>below-format-target,</code> <code>above-format-target</code>	Word count outside the assigned format range

B.5 SEO findings

Code	Severity and meaning
seo-ungrounded-keyword	Blocking — primary keyword unsupported by verified claims
seo-misleading-title	Blocking — SEO title unsupported by verified claims
seo-unsupported-superlative	Blocking — a ranking word with no verified basis
seo-keyword-stuffing	Blocking — over 4 repeats, or a secondary keyword duplicating the primary
seo-malformed-slug	Blocking — slug does not normalise to a usable URL
seo-missing-meta-description	Blocking — required metadata absent
seo-missing-primary-keyword	Blocking — required metadata absent
seo-meta-description-short / -long	Advisory — outside 50–160 characters
seo-title-long	Advisory — over 60 characters
seo-keyword-not-in-title	Advisory — placement
seo-keyword-not-in-heading	Advisory — placement (not applied to briefs)
seo-keyword-not-in-intro	Advisory — placement
seo-heading-not-grounded	Advisory — suggested heading has no verified material
seo-too-many-headings	Advisory — beyond the format's shape
seo-too-many-secondary-keywords	Advisory — beyond the format's shape
seo-intent-mismatch	Advisory — commercial intent declared for a brief

B.6 Tag rejections

Code	Meaning
empty, too-short, too-long	Length outside 2–40 characters
too-many-words	More than 3 words
control-characters	Contains a control character
markup	Contains angle brackets or an HTML entity
disallowed-characters	Outside the permitted character class
theme-not-entity	A theme, month or headline verb rather than a named entity
unslugifiable	Produces no usable WordPress slug

B.7 Deferrals — the story survives

Reason code	Meaning
candidate-budget-reached	Candidate cap reached before classification
verification-budget-reached	Verification cap reached
article-budget-reached	Article cap reached
llm-budget-exhausted	Model call or token budget exhausted
wordpress-not-configured	No CMS credentials; the approved draft is retained locally
wordpress-unavailable	The CMS could not be reached; the draft is retried on a later run
category-not-configured	The category does not exist in WordPress; never created automatically
taxonomy-resolution-failed	Term lookup failed for another reason

B.8 Publication guards — skipped, not failed

Reason code	Meaning
not-approved (<i>status</i>)	The draft never reached approved status
already-published (<i>wp:n</i>)	This article already has a WordPress post
duplicate-of-published-story	Merged into a story that already has a post
dry-run	A dry run was requested; no CMS request was made

Appendix C. Documented vs. Implemented Differences

Where the design brief (`NEWS_AGENT.md`) and the current code differ, this document describes the **code**. The differences are consolidated here.

Topic	Design brief says	Implementation does
Novelty score	Listed as a ranking signal alongside relevance and importance	Produced and schema-validated, but read by no gate or weight . The weighted score uses relevance, importance, source trust and freshness only
Core claim types	A core flag is configured per claim type (launch , capability , pricing)	No code reads the flag . The core claim is whichever claim the verifier nominates
Daily publication cap	<code>maxArticlesPublishedPerDay</code> listed among the configurable caps	Not implemented . The nearest control is <code>AGENT_MAX_DRAFTS_PER_RUN</code> — per run, not per day. Daily volume is governed by the schedule
Cap naming	<code>maxCandidatesScoredPerRun</code> , <code>maxArticlesGeneratedPerRun</code>	Implemented as <code>AGENT_MAX_CANDIDATES_PER_RUN</code> and <code>AGENT_MAX_ARTICLES_PER_RUN</code> — same semantics, different names
Source poll interval	<code>pollIntervalMinutes</code> appears in the source interface sketch	Not present on the implemented source type. Cadence is owned entirely by the external scheduler
Funding claims	“Tier 1 or Tier 2 (Reuters/Bloomberg class)”	One independent Tier 2 publisher suffices. There is no publisher-class filter within Tier 2, and no Reuters or Bloomberg feed is configured
Tier 1 examples	Anthropic, Meta, Mistral, Microsoft, Runway among others	All five are registered but disabled , each with a recorded reason (no stable feed). Reuters, Wired and Bloomberg are not registered at all
Tier 3 sources	Hacker News, Reddit, Product Hunt, GitHub Trending	None configured . Tier 3 remains the default tier for any unrecognised host, and its rules are exercised only by tests
Entity catalogue	“read a shared entity list” derived from the tool catalogue	Uses the curated 43-entry vocabulary in the agent’s own editorial configuration; the front-end catalogue is not imported
Article structure	Nine elements including an optional “AI Tool Kart take”	The writer is given five factual headings; the editorial-take section is not implemented
Format rule	Earlier text describes a flat 500–900 word minimum	Superseded in both the brief and the code by the three evidence-derived formats (Section 8)
SEO metadata	Written to the CMS	Local only . No safe REST write path exists for the agent’s role; the CMS sink is a deliberate no-op behind a one-file interface (Section 19)

How to read this table

None of these differences weakens a safety or quality gate. Two are inert configuration that the code does not consult (`novelty`, the `core` flag); four are naming or scope differences; four reflect sources that could not be enabled because no usable feed exists; and two record capabilities consciously deferred. They are listed so that a reader comparing this document against the design brief is not misled by either.

Compiled from the AI Tool Kart News Agent source. Every threshold, weight, gate and reason code in this document was read from the implementation; no parameter has been inferred or idealised.